

DIGITAL SYSTEMS ELECTRONICS

Academic year 2023-2024

LABORATORY NR. 5

DUE DATE: April 29, 2024
DELIVERY DATE: April 28, 2024

GROUP 03

CONTRIBUTIONS:

- El Amrani Dawid 100%
- Ielmini Giovanni 100%
- Di Carolo Pierfelice 100%
- Pavone Lorenzo 100%

The members of the group listed above declare under their own responsibility that no part of this document has been copied from other documents and that the associated code is original and has been developed expressly for the assigned project.

Contents

This lab experience aimed at investigating the operation of Finite State Machines, abstract machines that can be in exactly one of a finite number of states at any given time. We had to implement different FSMs with different approaches through VHDL coding.

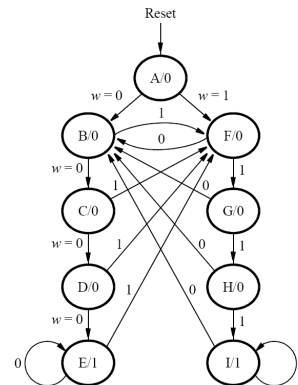
1. One-Hot Finite State Machine

a) Design entry:

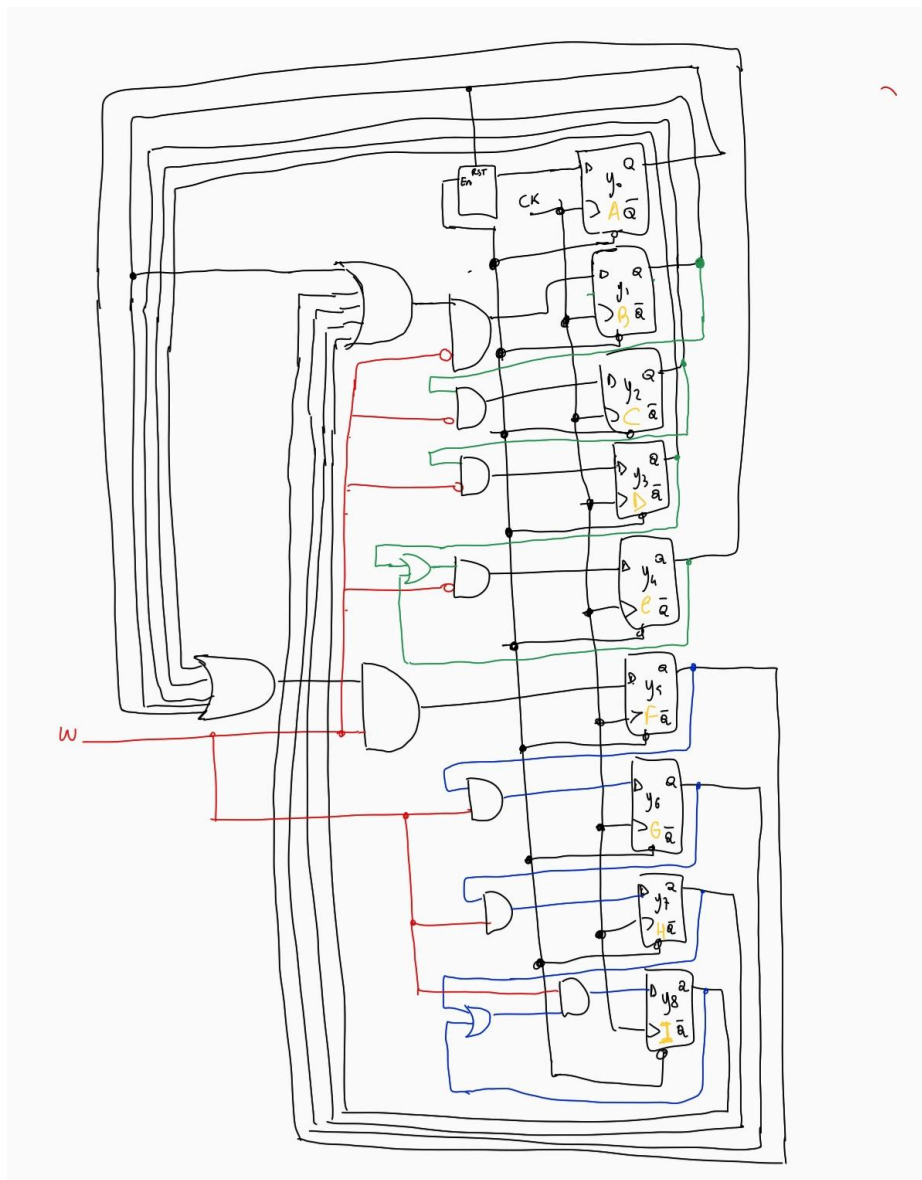
The first request was to implement an FSM able to recognize two specific sequences of input signals (four consecutive ones and four consecutive zeros).

Furthermore, we needed to follow a specific one-hot state assignment table:

Name	State Code
	$y_8y_7y_6y_5y_4y_3y_2y_1y_0$
A	000000001
B	000000010
C	000000100
D	000001000
E	000010000
F	000100000
G	001000000
H	010000000
I	100000000



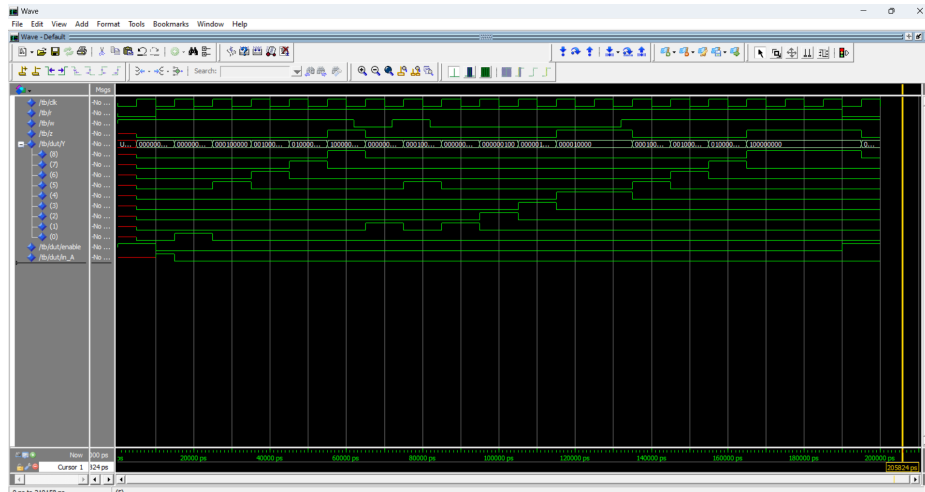
As requested, we drew the circuit before writing the code, and this is the scheme we based our code on:



The top level entity is “onehot_fsm” and it contains nine flip flops to represent the nine states of the FSM, with one additional flip flop (registerA) giving the right input to flip flop A when the circuit is resetted, and all the logic operations necessary to make the transitions between states correct.

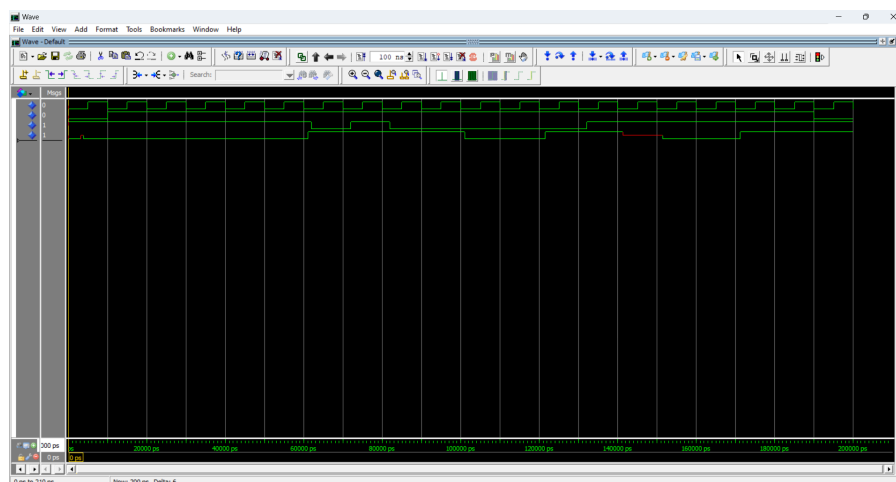
b) Functional simulation:

We tested the circuit with an ad-hoc testbench and this is the resulting waveform, turned out as we expected it:



We can appreciate how the state assignment table is observed, since only one of the flip-flops has high output at a time.

We ran the timing simulation as well, and it shows that the circuit has some delays when changing output with respect to the rising edge of the clock.



c) Synthesis:

The circuit worked correctly on the fpga.

2. Modified One-Hot Finite State Machine

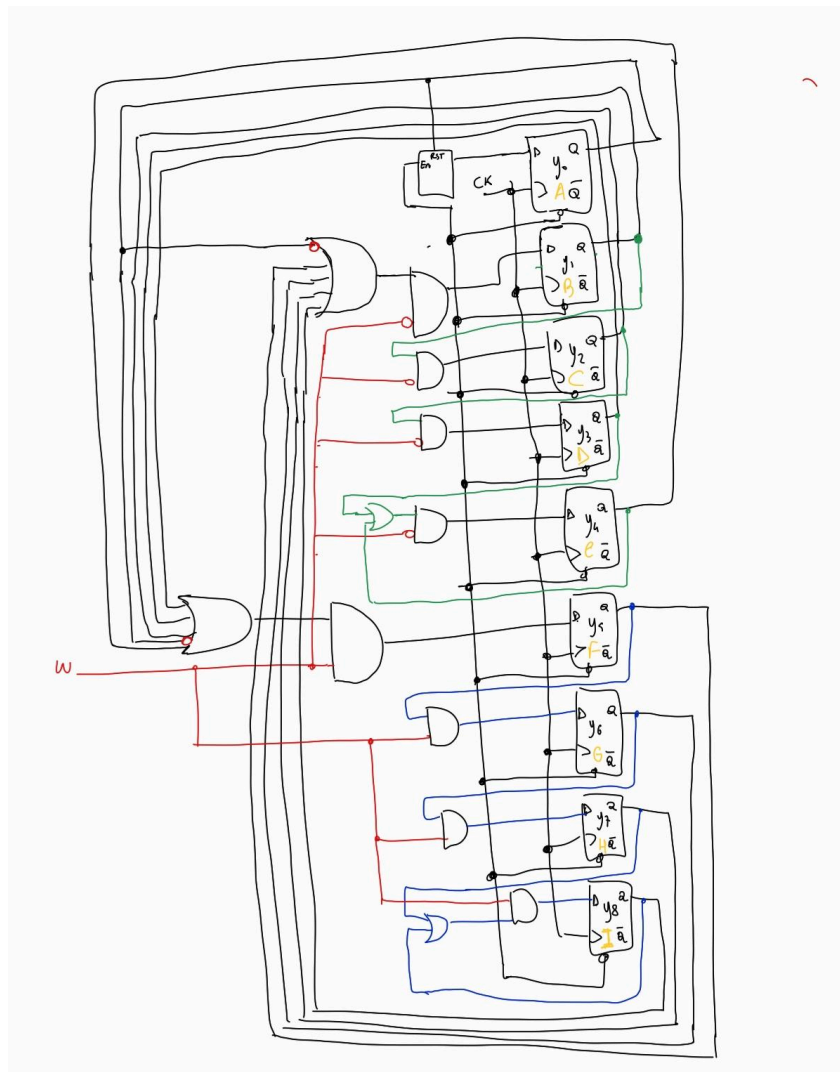
a) Design entry:

This circuit was almost equivalent to the first one, the only difference was in the state assignment table:

Name	State Code
A	000000000
B	000000011
C	000000101
D	000001001
E	000010001
F	000100001
G	001000001
H	010000001
I	100000001

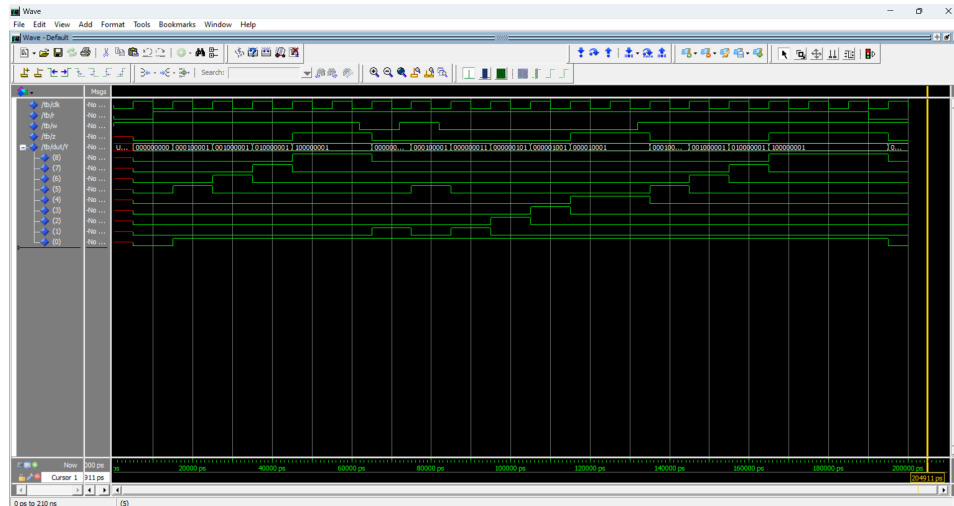
The request is basically to invert the state variable A ($y(0)$), to be able to get to the “000000000” state code when resetting.

We achieved this by making a few changes in the top level entity, adding a couple of inverters, precisely at the input of the OR gates leading to states B and F, and changing the job of registerA, to achieve a $y(0)$ always equal to 1, except when just resetted.

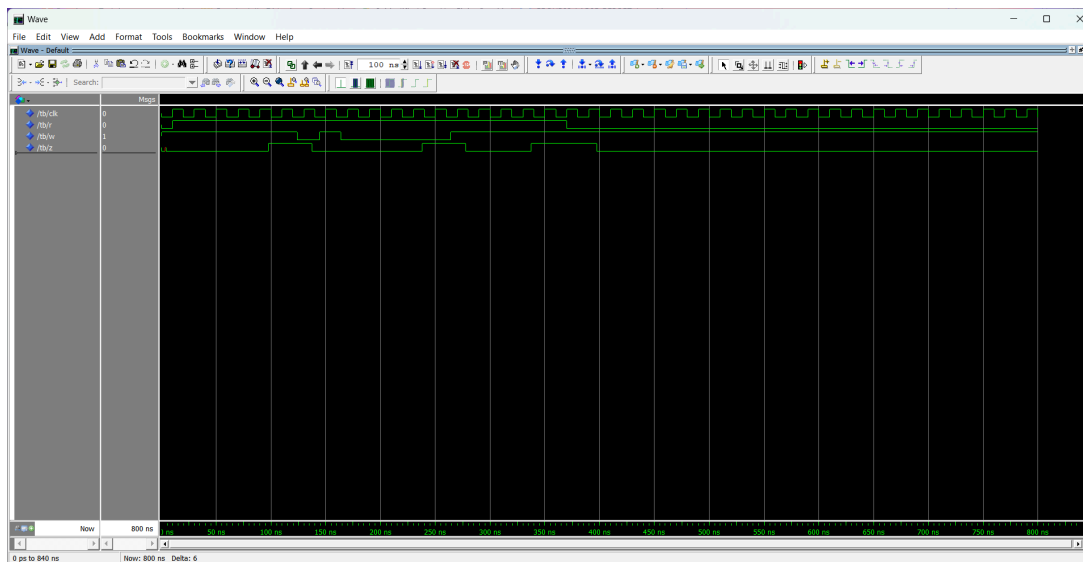


b) Functional simulation:

Using the same testbench as before, we can see how the only thing that changed is the state variable Y(0) relative to state A, that now is off when the circuit is in state A (or resetted) and on otherwise. This is the waveform we obtained:



Analyzing the timing simulation, we can easily detect the delays caused by the combinational parts of the circuit (the logic elements):



c) Synthesis:

The circuit worked correctly on the FPGA.

3. Two process FSM

a) Design entry:

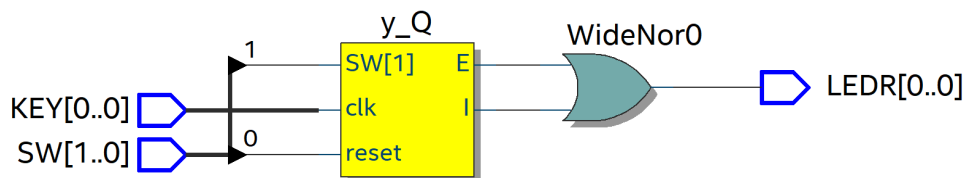
For the third time, we had to implement the circuit described in the first assignment of this laboratory experience, with the state table of assignment 2.

This time, though, we used another description approach: instead of the “one-hot” description, we went (as requested) for the “two-process” implementation.

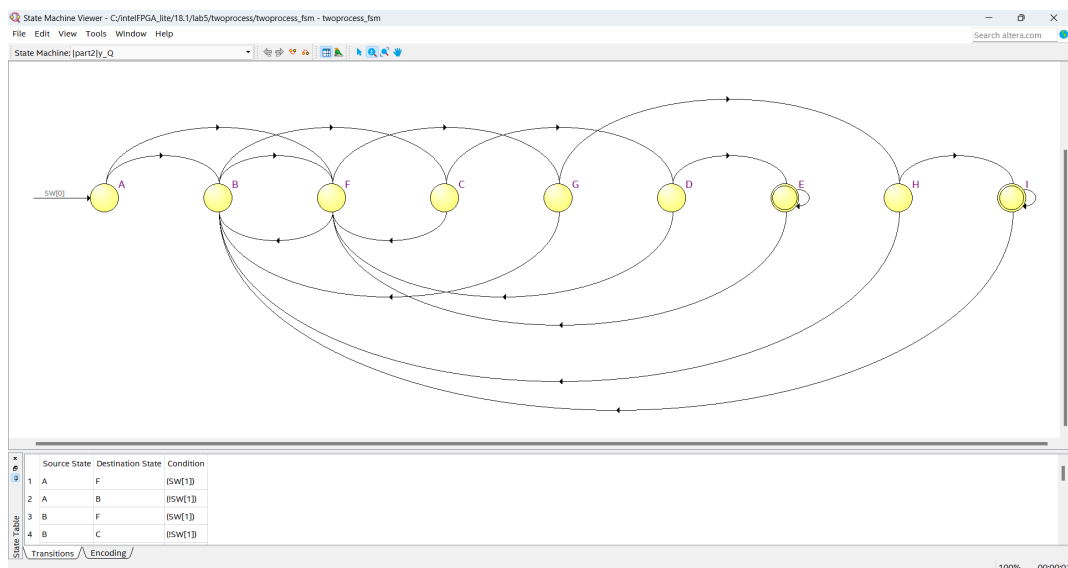
We actually used three processes in the top level entity: one for CC1, dedicated to the state assignment; one for MEM, relative to the flip-flops; and the last one for CC2, basically the output assignment.

Differently from the one-hot encoding, with this description strategy we don't need to implement the flip-flops, because the VHDL compiler itself decides how many are needed and allocates them automatically.

Using the RTL view (with minimal bits processing), this is the circuit obtained is:



And this is the view we get double clicking on the finite state machine. The state diagram designed by the compiler coincides with our starting diagram.



Checking (with one-hot processing) the state table, we notice there are no differences with the one we wanted to implement:

State Machine - |part2|y_Q

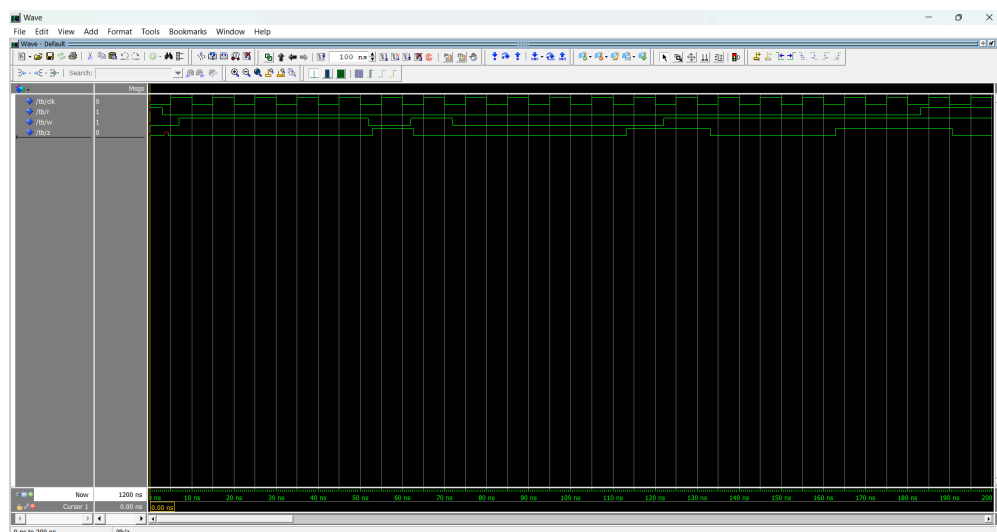
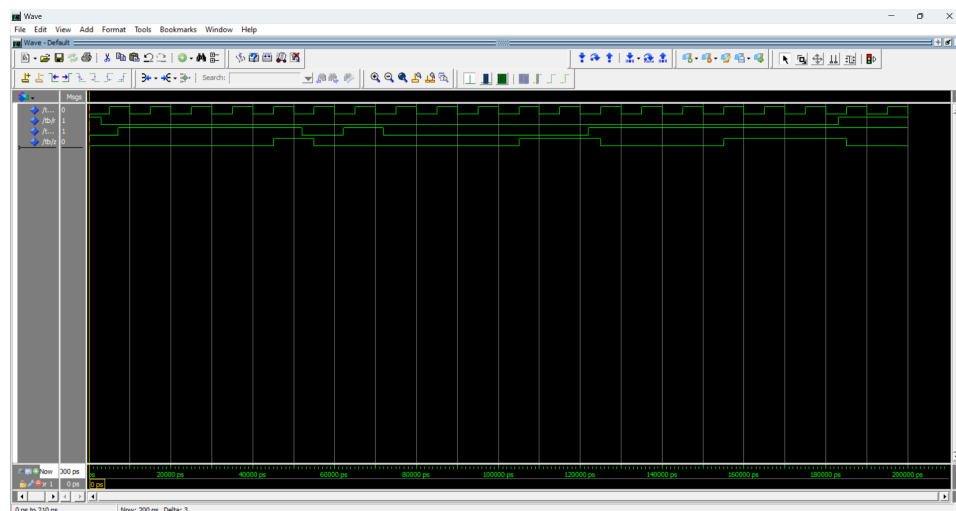
<<Filter>>

	Name	y_Q.I	y_Q.H	y_Q.G	y_Q.F	y_Q.E	y_Q.D	y_Q.C	y_Q.B	y_Q.A
1	y_Q.A	0	0	0	0	0	0	0	0	0
2	y_Q.B	0	0	0	0	0	0	0	1	1
3	y_Q.C	0	0	0	0	0	0	1	0	1
4	y_Q.D	0	0	0	0	0	1	0	0	1
5	y_Q.E	0	0	0	0	1	0	0	0	1
6	y_Q.F	0	0	0	1	0	0	0	0	1
7	y_Q.G	0	0	1	0	0	0	0	0	1
8	y_Q.H	0	1	0	0	0	0	0	0	1
9	y_Q.I	1	0	0	0	0	0	0	0	1

b) Functional simulation:

To virtually test the functioning of this FSM, we used the same tb as the first two assignments, since we are dealing with the same circuit.

In the pictures below are shown the regular simulation and the timing simulation, in which we can notice some delays due to the combinational components (CC1 and CC2) of the FSM.



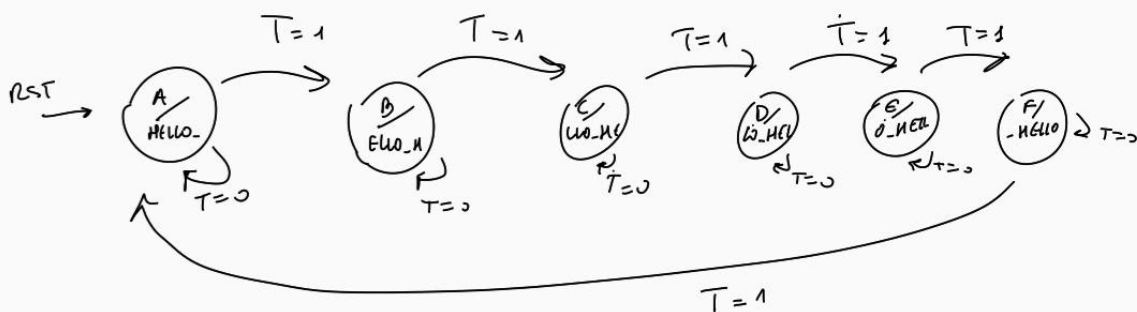
c) Synthesis:

The circuit worked correctly when loaded on the FPGA board.

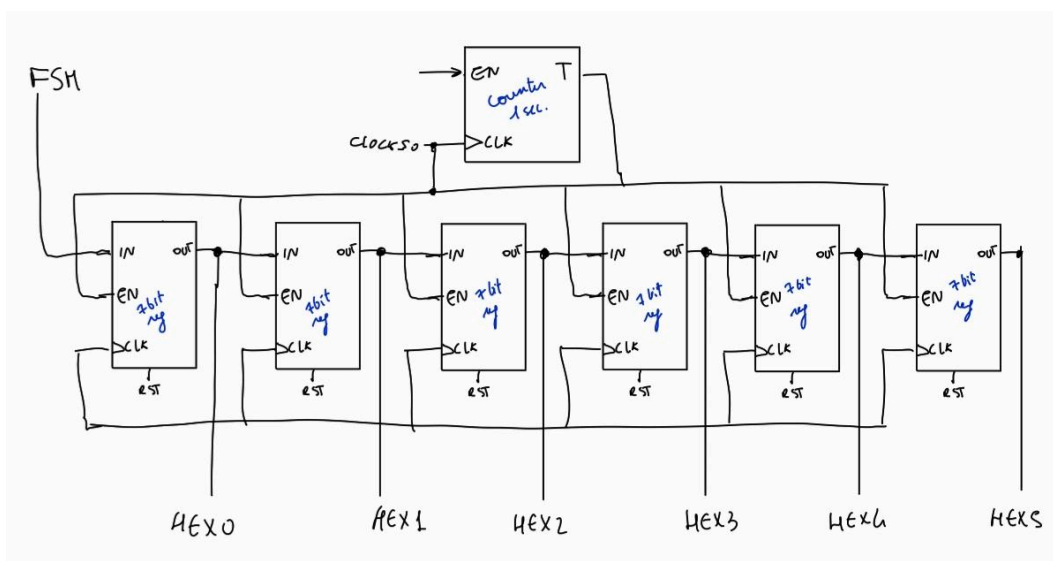
4. "HELLO" FSM

a) Design entry:

As the last assignment, we had to design a circuit displaying the word HELLO and making it scroll from right to left. In the picture below we see the state diagram of the FSM. We have six states, one for each configuration of the letters on the displays. If the time elapsed is equal to one second we go to the next state, otherwise we stay there. After we've reached the last state (state F), if the time elapsed is equal to one second we go again to the first one (state A). When we reset the circuit we start from state 1.



The block diagram for the design is in the picture below. We have a one second counter which controls the 'enable' input of the six 7-bit registers. The first input is derived from the FSM which gives the input the letter when we start the circuit. The inputs of the other registers are connected to the output of the previous one and the output of every register is connected to the display.



Our FSM machine is described in the "hello_fsm" file, which is the top-level entity. We have the reset as a KEY input, a switch to turn on the circuit, CLOCK_50 as clock input, and then the six HEX displays as output.

The “counter1s” is the same design we already used in the previous lab and which we have already described, it’s simply a counter that gives ‘1’ as output when one second is elapsed. We also are reusing the classic “flipflop” design with a synchronous reset which is called seven times in the 7-bit register called “bit7reg”. The 7-bit reg memorizes the number that we need to display while we are in the current state to then pass it to the next register when one second has elapsed.

In the top-level entity (“hello_fsm”) we have defined a new type for the states, each one of them is relative to a letter/space, and then we have the looping state when we have to loop our word:

TYPE STATE IS (SPACE, O, L1, L2, E, H, LOOPING);

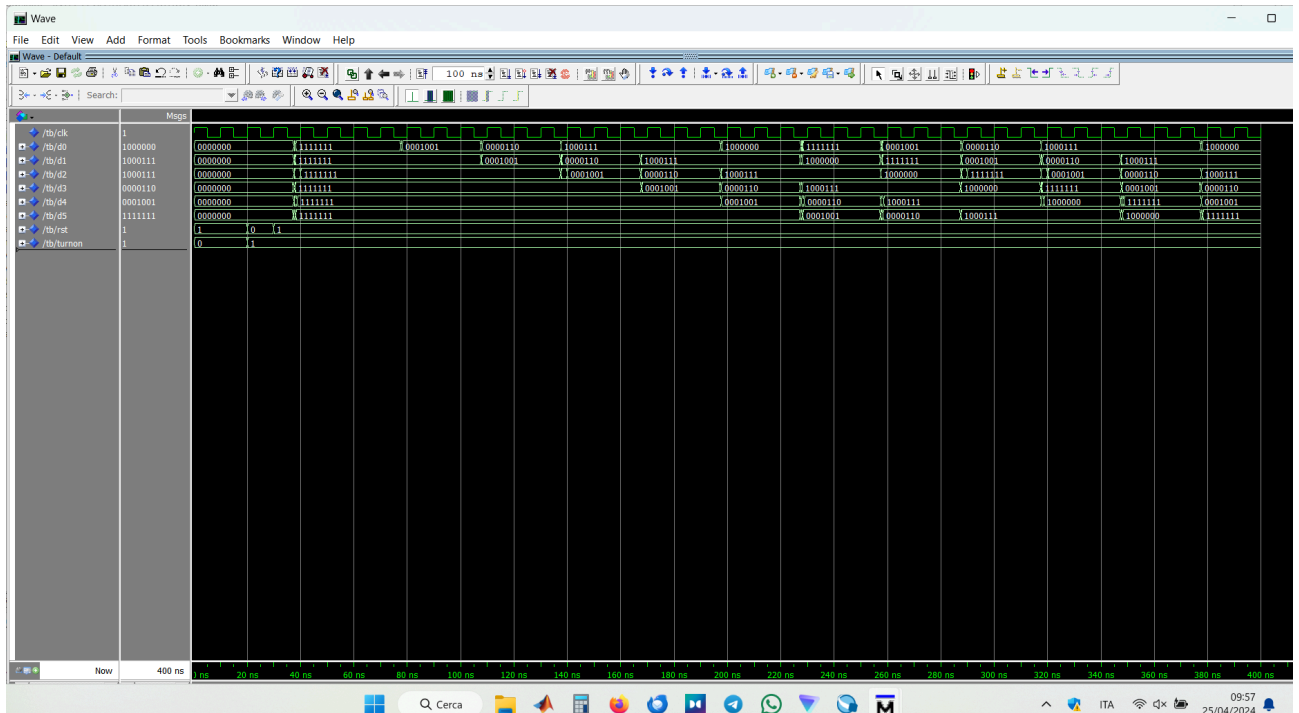
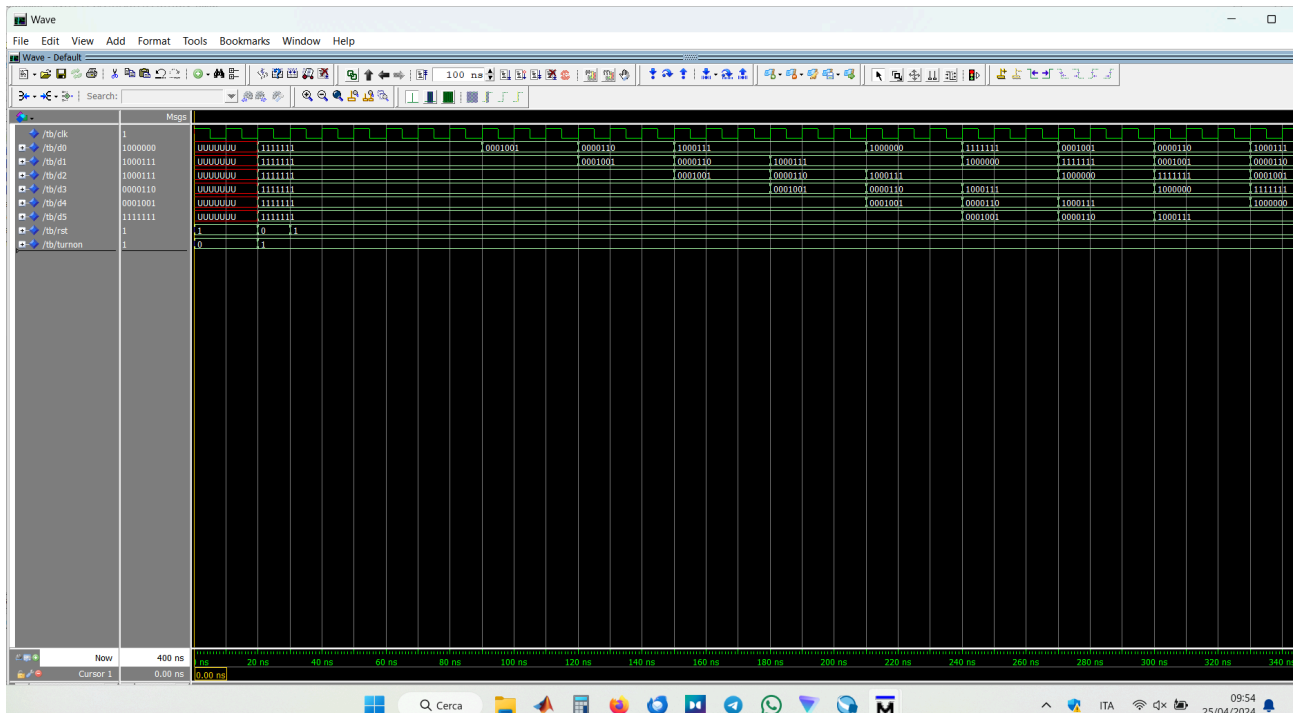
and we’ve used this type for signals X and Y which are respectively the current and future state. The file has three processes. The first one (called “restart”) is used to reset the circuit when the key is pressed and checks if we have this condition on every rising edge of the 50 MHz clock signal (CLOCK_50).

Then we have the “state_assignment” process which, using the CASE statement, assigns the future state based on the current state and if the “enable” signal is equal to one. The enable signal is the output of the “counter_1s” which we call in this file to know when one second is elapsed. This process is activated only when “enable” changes.

The last process is the “letter_assignment” and is used to assign to the signal that is going into the register the correspondent 7-segment code of the letter. To memorize the current state we just call every 7-bit register feeding the IN0 signal to the first one and then feeding the output of every register to the input of the next one. Then we just assign to the displays (HEX) the output of every register.

b) Functional simulation:

To test the functionality of our design we built a testbench (‘tb’) which simply sets the clock cycle with a 10 ns period and the reset and enable signals. As shown in the first picture below it works perfectly. The second picture shows the timing simulation result which is positive as no problem is introduced by the delays.



- c) **Synthesis:**
The circuit worked correctly when loaded on the FPGA.